

Лабораторная работа 3

Разработка приложения БД

Цель: Изучить основы проектирования приложений баз данных с использованием СУБД типа SQL. Освоить методику просмотра и редактирования таблиц в оконном и табличном интерфейсе, отбор записей таблиц

Часть 2. Просмотр и редактирование простой таблицы

2.1 Просмотр и редактирование данных

Схема базы данных, которая будет использована в программном приложении представлена на рис.1.

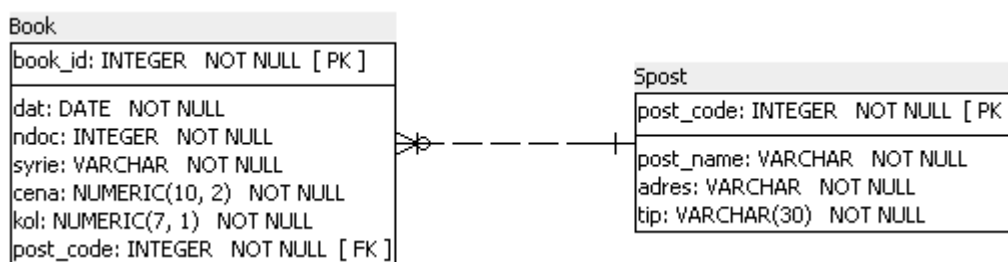


Рис.1 Схема БД

Начнем с таблицы справочника поставщиков **Spost**. Данные этой таблицы будем получать из БД и передавать их для отображения в визуальные компоненты. Также рассмотрим процесс редактирование данных с передачей изменений для сохранения в БД.

Данные справочника будем отображать в табличном формате, а добавлять и редактировать строки справочника будем в отдельной полноэкранной форме (рис.2).

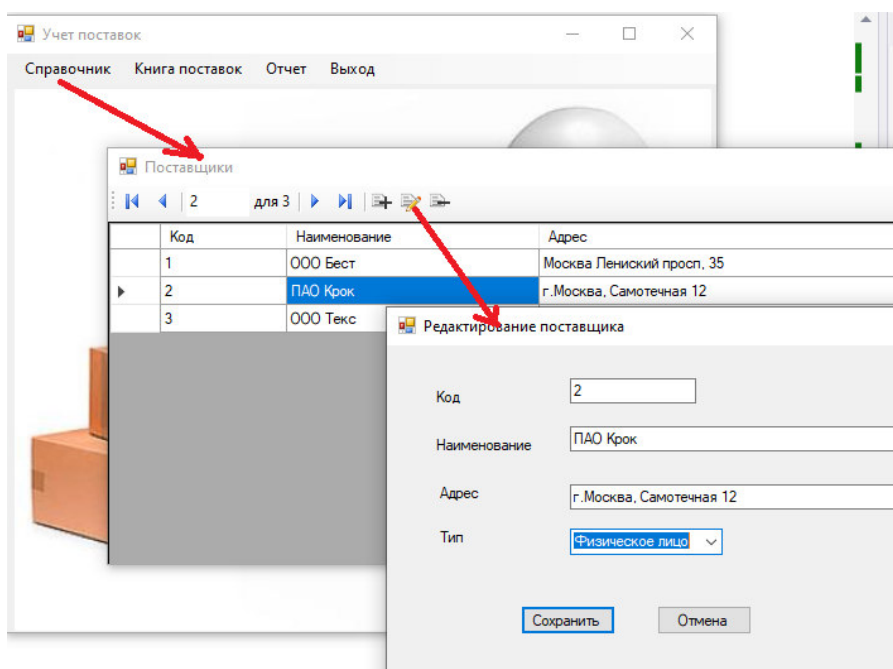


Рис. 2 Просмотр и редактирование справочника поставщиков

Рассмотрим создание новых форм и способы их вызова.

2.2 Создание формы просмотра справочника

Создаем форму просмотра справочника **Spost**.

Создадим новую форму, выбрав пункт меню

Проект – Добавить форму (Windows Forms)

Выберем шаблон создаваемой формы **Windows Forms**, зададим имя формы **FrmSpost**.

На форму добавим таблицу **DataGridView** (рис.3). Задаем элементу имя **dgSpost**. Над таблицей разместим элемент навигатора **BindingNavigator**. Задаем ему имя **bnSpost**. Начнем с настройки таблицы **DataGridView**.

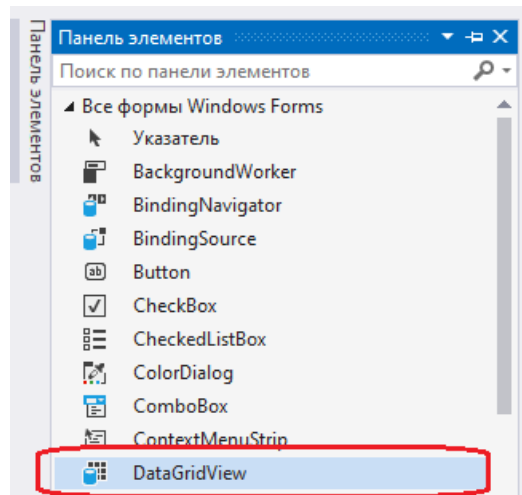


Рис.3 Выбор табличного элемента

Зададим источник данных, которым будет выступать таблица базы данных **Spost**. Для задания источника щелкаем по маленькой кнопке в правом верхнем углу элемента, раскроем список *Выберите источник данных* и щелкнем по ссылке *Добавить источник данных проекта* (рис.4).

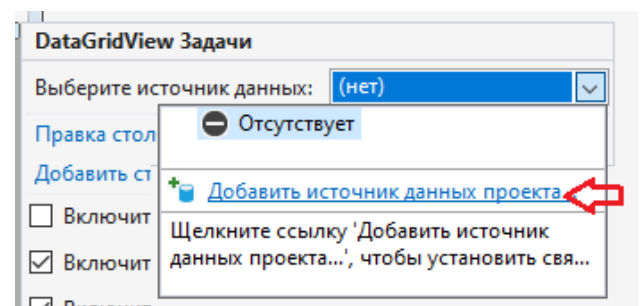
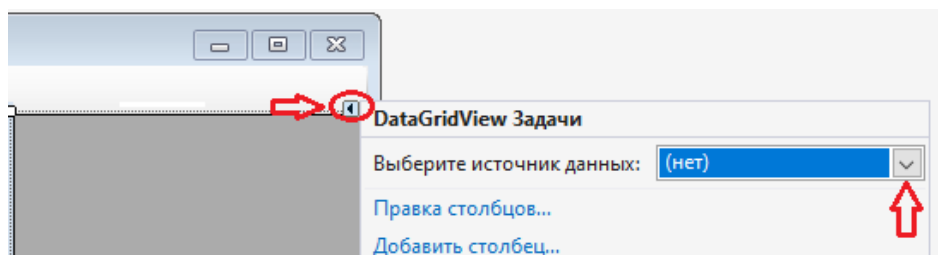


Рис. 4. Добавление источника данных

В качестве источника выбираем *Базу данных* и нажимаем *Далее*. В следующем окне выбираем *Набор данных* и нажимаем *Далее* (рис.5).

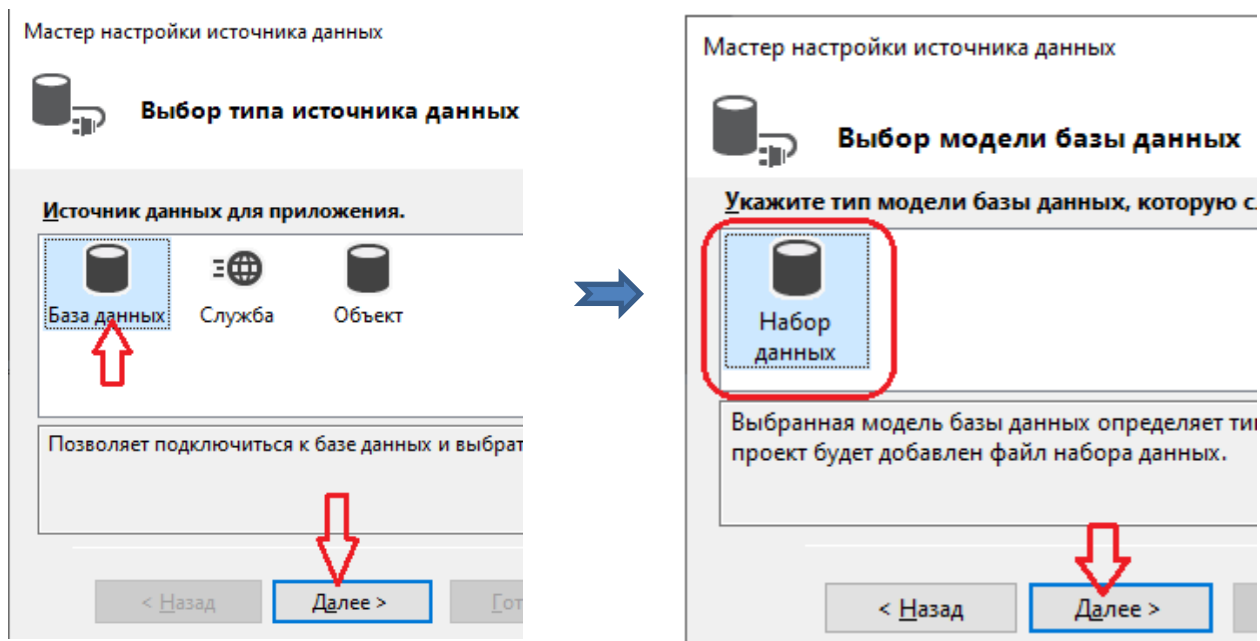


Рис. 5 Выбор источника данных

На следующем шаге нажимаем кнопку *Создать подключение*. В открывшемся окне выбираем *Источник данных Microsoft ODBC* и нажимаем кнопку *Продолжить* (рис.6).

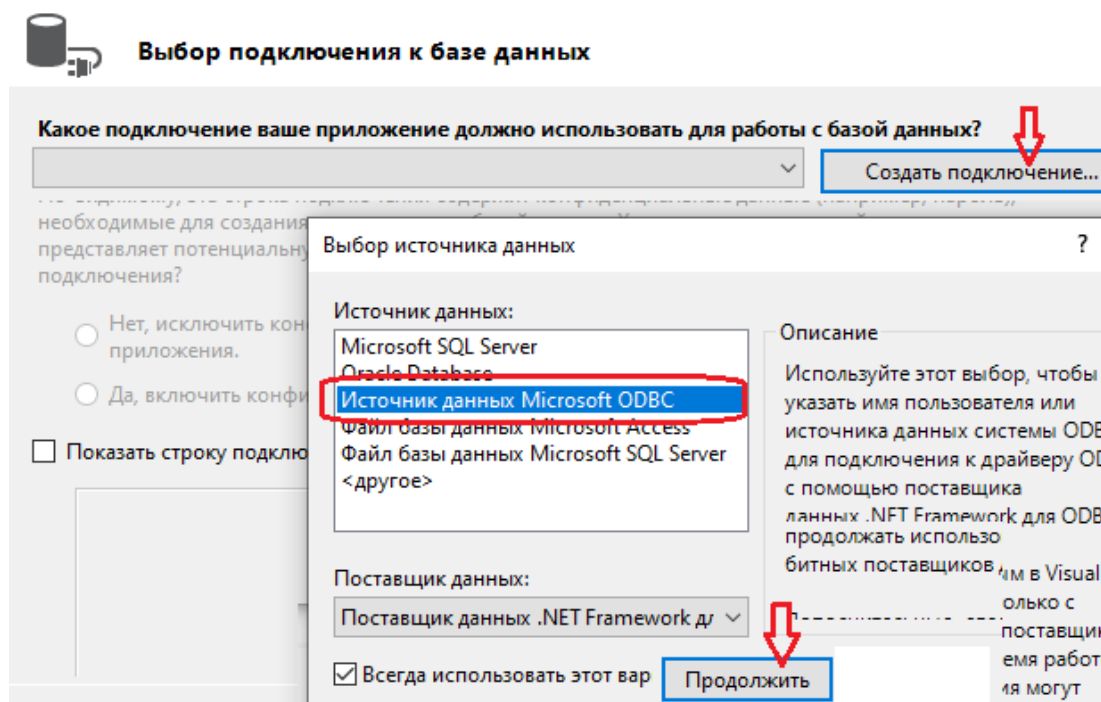


Рис. 6. Создание подключения

Выбираем источник (созданный в лабораторной работе 2), вводим данные авторизации (имя пользователя и пароль) и нажимаем кнопку *Проверить подключение* (рис.7). Если все выполнено правильно, то будет выведено окно с сообщением о выполнении проверки подключения.

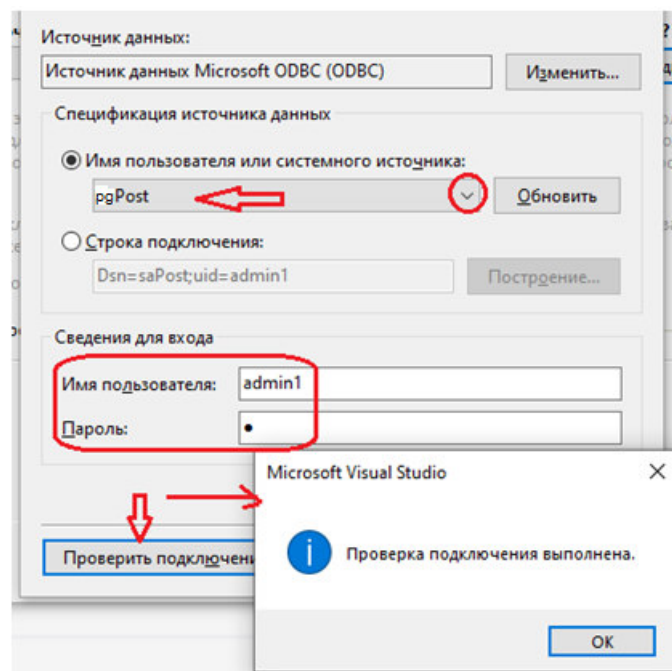


Рис. 7. Задание параметров подключения

Устанавливаем необходимые переключатель и флаг как показано на рисунке 8 и нажимаем кнопку *Далее*.

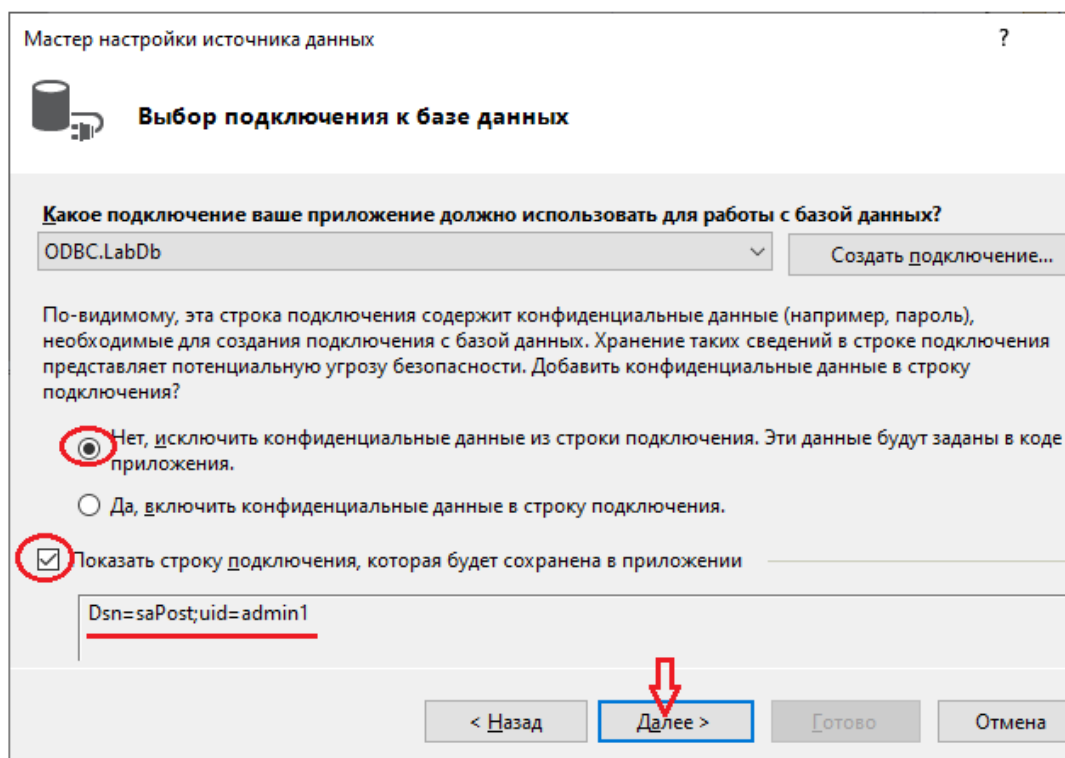


Рис.8. Задание дополнительных параметров

В следующем окне будут выведены таблицы базы данных. В списке таблиц выбираем справочник поставщиков (таблицу **SpPost**). Указываем имя набора данных **dsSpPost** и нажимаем *Готово* (рис. 9).

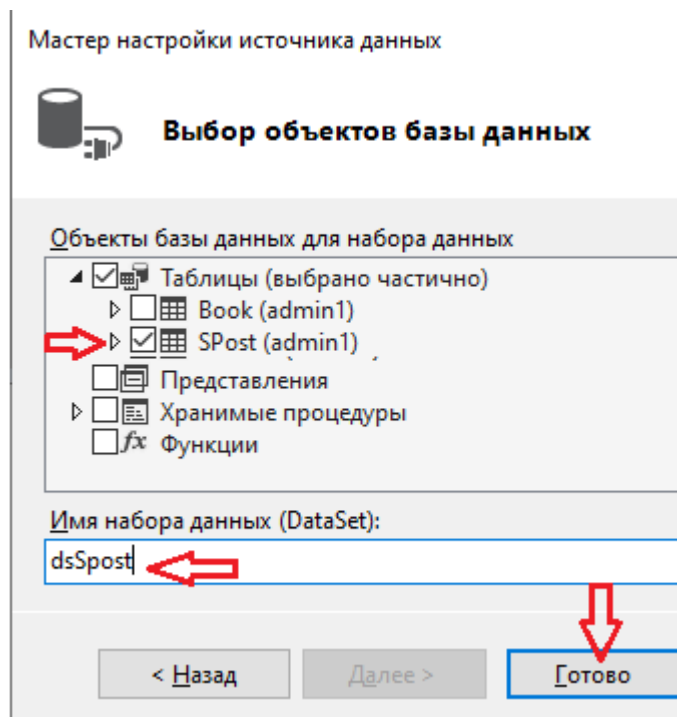


Рис. 9. Выбор таблицы

Примечание. Если при нажатии кнопки *Готово* возникает ошибка:

«Не удалось загрузить файл или сборку microsoft.data.sqlclient version 5.0.0.0 либо одну из зависимостей. Не удастся найти заданный файл...»,

то необходимо подключить к среде разработки библиотеку

Microsoft.Data.SqlClient.

Файл с настроечными параметрами проекта **Settings.settings** с добавленным подключением приведен на рис. 10.

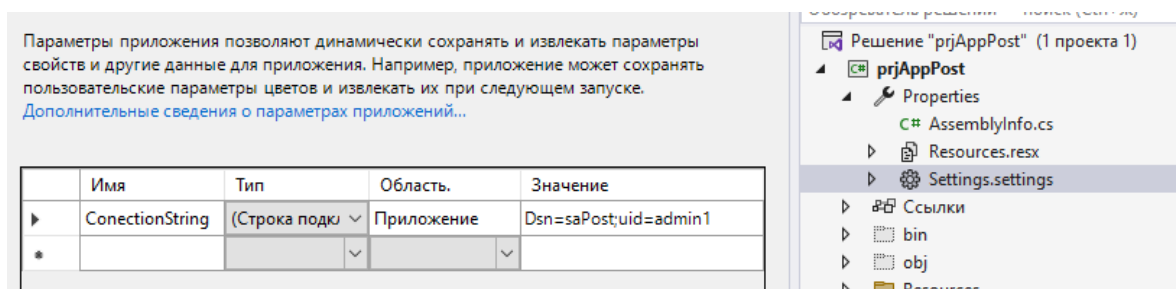


Рис. 10. Настроенное соединение

Данное соединение будем использовать только для настройки таблицы **DataGridView**. Во время работы приложения будем использовать источник соединения, открытый в процессе авторизации.

Подготовленный элемент таблицы **DataGridView** представлен на рисунке 11.



Рис. 11. Подготовленная таблица

Изменим название колонок. Для этого на заголовке колонок таблицы в контекстном меню выберем пункт *Правка столбцов* (рис. 12).

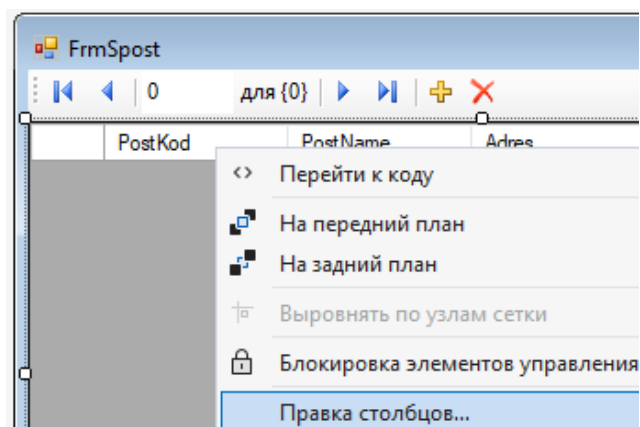


Рис. 12 Переход в редактор столбцов таблицы

В открывшемся окне редактора (рис.13) необходимо последовательно в левой панели выбирать столбец, а в правой панели свойств изменить свойства выбранного столбца. Заголовок столбца задается свойством **HeaderText**, ширина колонки – свойством **Width**, шрифт - свойством **Font**.

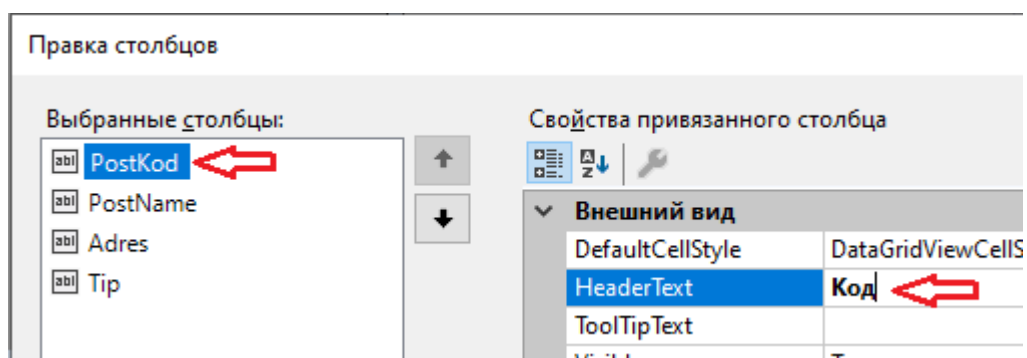


Рис. 13. Настройка свойств столбцов

Настроим стиль отображения заголовочной части таблицы (названия колонок). Для этого в списке элемента найдем свойство **ColumnHeadersDefaultCellStyle** (рис. 14) и настроим подсвойства **Font** и **Alignment** (рис. 15).

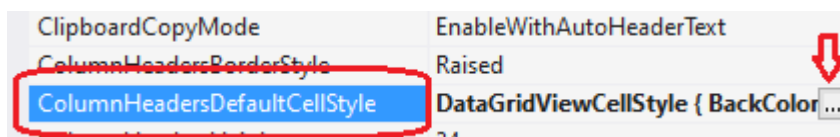


Рис. 14. Свойство настройки шапки таблицы

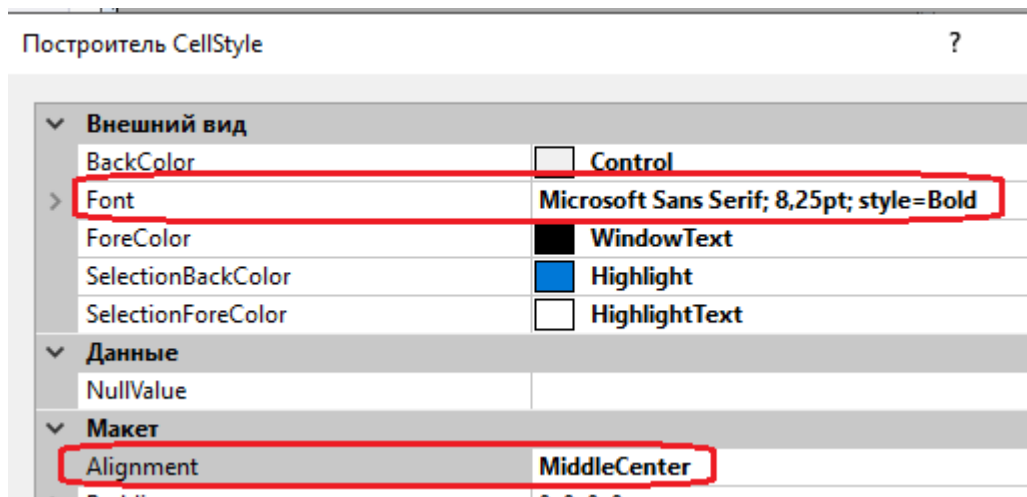


Рис. 15. Настройка свойств заголовка

После внесенных изменений таблица будет выглядеть, как показано на рисунке 16.

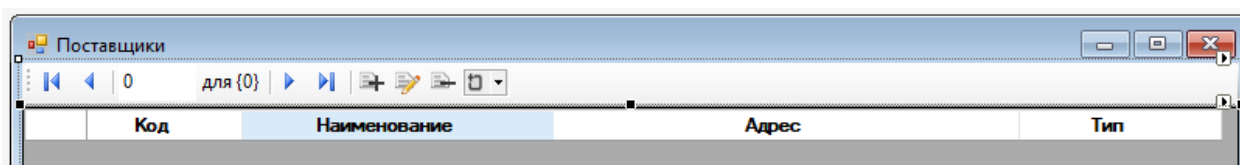


Рис. 16. Отформатированный заголовок таблицы

Перейдем к настройке элемента навигатора **BindingNavigator**, расположенного над таблицей. Отключим свойства (удалим) добавления **addNewItem** и удаления **DeleteItem**. Добавим на элемент три новые кнопки с именами (свойство Name) **sbAdd**, **sbEdit**, **sbDel** (рис. 17). Для кнопок в свойстве **Image** загрузим изображение (рис. 18).

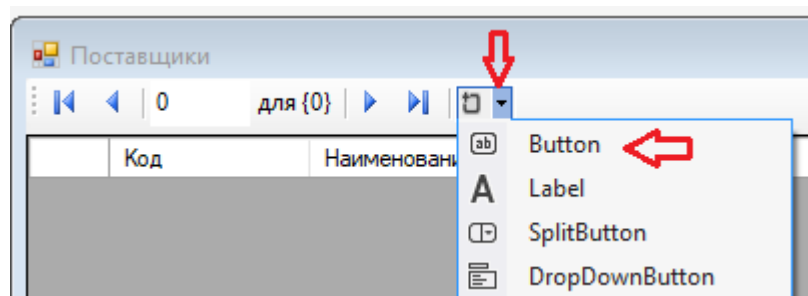


Рис. 17. Добавление кнопки в навигатор



Рис. 18. Добавленные кнопки с загруженными изображениями

Привяжем элемент **DataGridView** к границам окна, предварительно растянув их к граница окна. Далее задаем свойство **Anchor=Top,Left,Right,Bottom** (рис. 19). После этого элемент таблицы будет всегда занимать всю область окна формы даже при изменении размера окна.

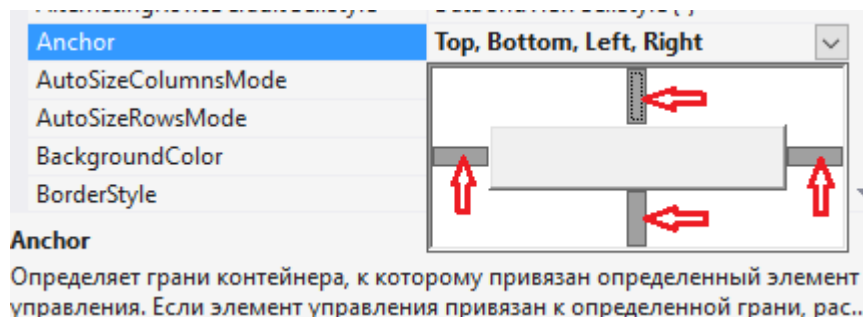


Рис. 19. Настройка свойства привязки к границам Anchor

После настройки пользовательского интерфейса перейдем к программному коду.

Открытие формы просмотра справочника свяжем с событием **Click** пункта меню Справочник в главной форме **FrmMain** (рис. 20).

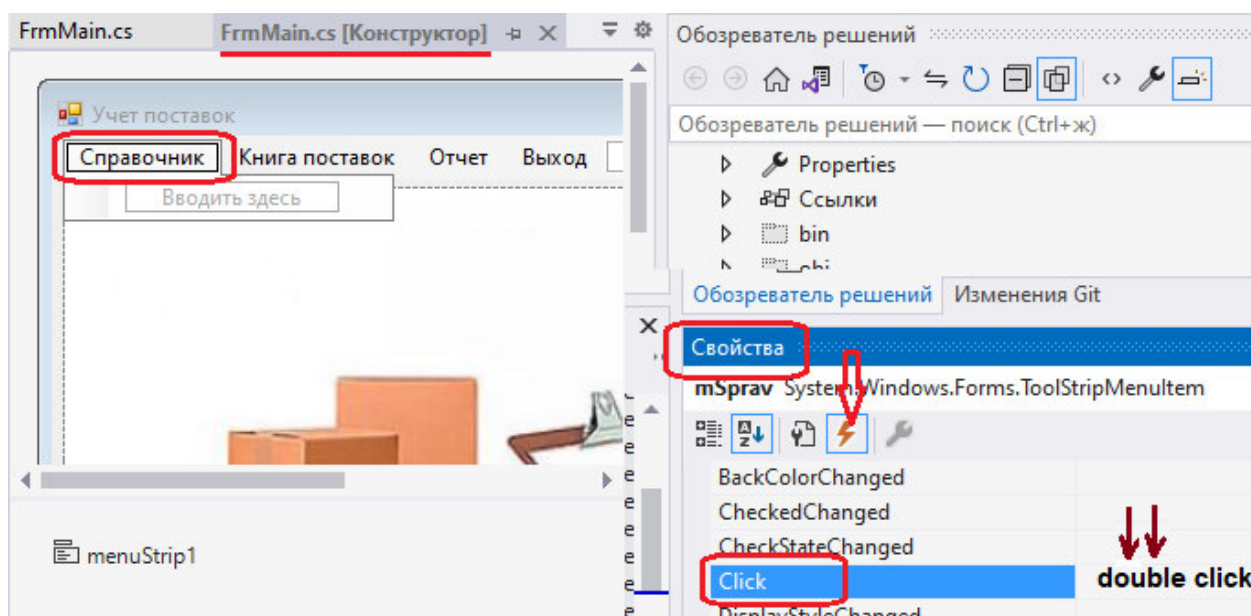


Рис. 20. Создание обработчика пункта меню Справочник

Программный код обработчика пункта меню:

```
private void mSprav_Click(object sender, EventArgs e) {
    FrmSpost frm = new FrmSpost();
    try {
        frm.ShowDialog();
    } finally {
        frm.Dispose();
    }
}
```

Переключимся в форму просмотра справочника и добавим в нее программный код. Загрузку данных в компонент таблицы привяжем к событию загрузки окна формы **Load**.

Код загрузки данных оформим в отдельном методе **loadData**.

```
private void loadData() {
    // Текст оператора запроса
    string strSql = "select * from имя_схемы.spost";
```



```

// Используем строку соединения, заданную при авторизации
var conStr = FrmMain.connectionString;
// Используем класс адаптера для получения данных
// Он будет источником данных для таблицы и навигатора
using (OdbcDataAdapter dadapter =
        new OdbcDataAdapter(strSql, conStr)) {
    DataTable table = new DataTable();
    dadapter.Fill(table);
    BindingSource bs = new BindingSource();
    bs.DataSource = table.DefaultView;
    this.bnavSpост.BindingSource = bs;
    this.dgSpост.DataSource = bs;
}
}

```

В обработчике загрузки окна будем вызывать этот метод.

```

private void FrmSpост_Load(object sender, EventArgs e) {
    loadData();
}

```

Формирование и выполнение запрос для удаления строки справочника свяжем с событием **Click** кнопки **sbDel**, размещенной на элементе навигатора (рис. 21).

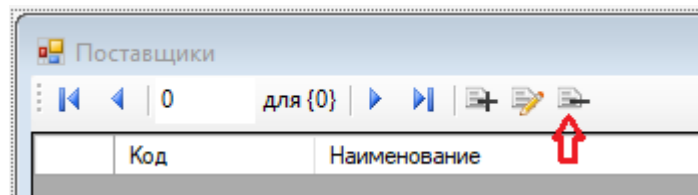


Рис.21. Задание обработчика для кнопки удаления строки

Программный код обработчика кнопки удаления строки:

```

private void sbDel_Click(object sender, EventArgs e) {
    // Вывод диалога подтверждения удаления строки
    DialogResult dr = MessageBox.Show("Удалить строку?",
        "Удаление", MessageBoxButtons.OKCancel,
        MessageBoxIcon.Question);
    if (dr != DialogResult.OK) return;
    // Определяем индекс текущей строки
    int ind = dgSpост.CurrentCell.RowIndex;
    // Определяем ключ удаляемой строки (колонка с индексом 0)
    string key = dgSpост.Rows[ind].Cells[0].Value.ToString();
    // Формирование запроса удаления строки
    string strSql = "";
    strSql += "DELETE FROM имя_схемы.spост ";
    strSql += " WHERE post_code=" + key;
    // Используем соединение, полученное при авторизации
    OdbcConnection con = FrmMain.odbcCon;
    // Создание объекта запроса
    OdbcCommand cmd = new OdbcCommand(strSql, con);
    // Выполнение запроса
    try {
        cmd.ExecuteNonQuery();
    }
}

```

```

    } catch (Exception ex) {
        MessageBox.Show("Ошибка удаления строки \n"+ex.Message);
        return;
    }
    // Перегружаем данные таблицы
    loadData();
    // Устанавливаем курсор таблицы на предыдущую строку
    if (ind > 0) ind--;
    dgSpост.CurrentCell = dgSpост[0, ind];
}

```

Программный код формирования и отправки запросов редактирование данных справочника рассмотрим после создание новой формы редактирования.

2.3 Создание полноэкранной формы редактирования справочника

Добавим новую форму в проект и дадим ей имя (свойство **Name**) **FrmEdSpост**.
Зададим свойство расположения формы **StartPosition= CenterParent**.
Добавим на форму элементы (рис. 22):

- 4 элемента статического текста **Label**
- 3 элемента полей ввода **TextBox** (**Name=edKod, edName, edAdr**)
- 1 элемент выпадающего списка **ComboBox** (**Name=cbTip**)
- 2 элемента кнопок **Button** (**Name=bOk, bCancel**)

Рис. 22. Форма редактирования

Для добавленных полей **edKod, edName, edAdr, cbTip** в файле **FrmEdSpост.Designer.cs** изменим модификатор доступа с **private** на **public**.

Для элемента **ComboBox** настроим свойство **Items**, в котором добавим элементы списка (рис. 23).

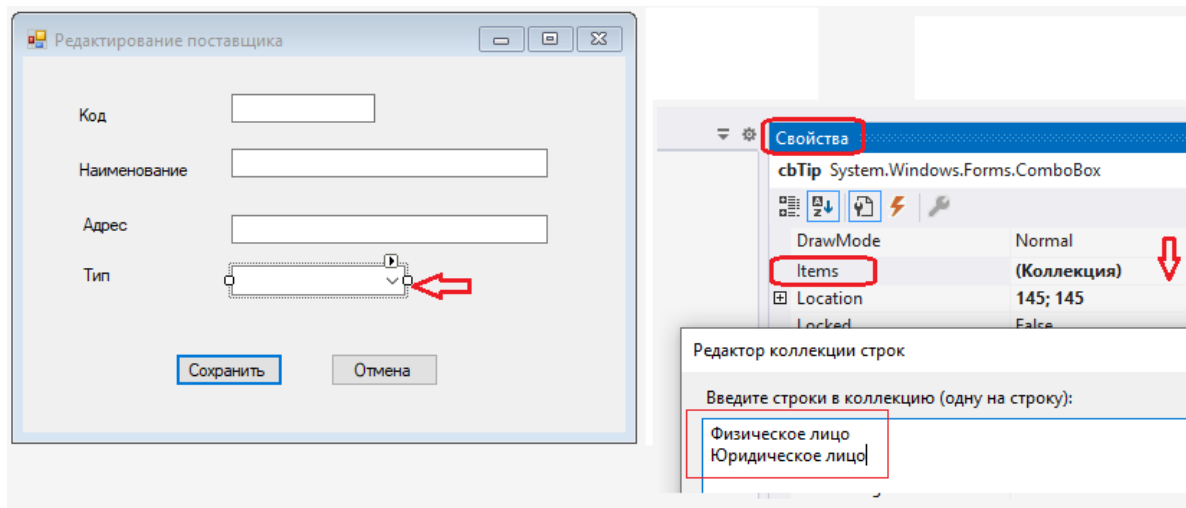


Рис. 23. Добавление элементов выпадающего списка

Для кнопок зададим свойства (таблица 1).

Таблица 1. Свойства кнопок формы редактирования

Свойство	Кнопка 1	Кнопка 2
name	bOk	bCancel
Text (надпись)	Сохранить	Отмена
DialogResult (код завершения)	-	Cancel

В класс формы добавим 2 поля:

- режим работы формы (добавления или редактирования)
- значение ключа редактируемой записи (для режима редактирования)

```
private bool isNew = true;
private String old_key;
```

В конструктор окна будет передавать 2 параметра, которыми инициализируются поля класса:

```
public FrmEdSpost(bool isNew, String old_key) {
    this.isNew = isNew;
    this.old_key = old_key;
    InitializeComponent();
}
```

Форма будет открываться при нажатии кнопок навигатора в форме просмотра (рис.24).

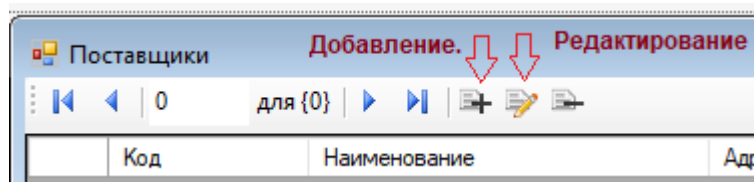


Рис. 24. Кнопки добавления и редактирования

Программный код обработчика кнопки вызова формы редактирования в режиме добавления новой строки (нового поставщика):

```
private void sbAdd_Click(object sender, EventArgs e){
```

```

FrmEdSpost frm = new FrmEdSpost(true, "");
// Заголовок формы в режиме добавления нового поставщика
frm.Text = "Добавление поставщика";
frm.ShowDialog();
// Перегружаем данные в случае добавления новой строки
if (frm.DialogResult == DialogResult.OK) {
    loadData();
}
frm.Dispose();
}

```

Программный код обработчика кнопки вызова формы редактирования в режиме редактирования существующей строки:

```

private void sbEdit_Click(object sender, EventArgs e) {
    // Запоминаем номер текущей строки
    int ind = dgSpost.CurrentCell.RowIndex;
    // Создаем форму (второй параметр – ключ редактируемой строки)
    FrmEdSpost frm = new FrmEdSpost(false,
        dgSpost.Rows[ind].Cells[0].Value.ToString());
    // Заголовок формы в режиме редактирования
    frm.Text = "Редактирование поставщика";
    // Заполняем поля ввода текущими значениями
    frm.edKod.Text = dgSpost.Rows[ind].Cells[0].Value.ToString();
    frm.edName.Text = dgSpost.Rows[ind].Cells[1].Value.ToString();
    frm.edAdr.Text = dgSpost.Rows[ind].Cells[2].Value.ToString();
    frm.cbTip.Text = dgSpost.Rows[ind].Cells[3].Value.ToString();
    frm.ShowDialog();
    // Перегружаем данные в случае сохранения изменений
    if (frm.DialogResult == DialogResult.OK) {
        loadData();
        dgSpost.CurrentCell = dgSpost[0, ind];
    }
    frm.Dispose();
}

```

Добавим обработчики кнопок формы редактирования.

При нажатии кнопки **Отмена** необходимо просто закрыть форму. При этом в форму просмотра будет передан код возврата **DialogResult=Cancel**.

Программный код обработчика кнопки отмены изменений:

```

private void bCancel_Click(object sender, EventArgs e) {
    Close();
}

```

При нажатии кнопки **Сохранить** необходимо сформировать и отправить на сервер БД запрос с командой изменения данных. В режиме добавления на сервер будет послана команда INSERT, а в режиме редактирования – команда UPDATE. В случае успешного выполнения команды, форма закрывается, а в форму просмотра будет передан код возврата **DialogResult=OK**. При возникновении ошибки в ходе выполнения программы будет выведено сообщения и форма редактирования закрыта не будет.

```

private void bOk_Click(object sender, EventArgs e) {
    string strSql = "";
    if (isNew) {
        strSql += "INSERT INTO имя_схемы.spost(post_code,post_name,adres,tip)
        values(";
    }
}

```

```

        strSql += edKod.Text;
        strSql += "," + edName.Text + "'";
        strSql += "," + edAdr.Text + "'";
        strSql += "," + cbTip.Text + "'";
    } else {
        strSql += "UPDATE имя_схемы.spost SET ";
        strSql += "post_code="+edKod.Text;
        strSql += ",post_name='" + edName.Text + "'";
        strSql += ",adres='" + edAdr.Text + "'";
        strSql += ",tip='" + cbTip.Text + "'";
        strSql += " WHERE post_code=" + old_key;
    }
    String conStr = FrmMain.connectionString;
    OdbcConnection con = FrmMain.odbcCon;
    OdbcCommand cmd = new OdbcCommand(strSql, con);
    try {
        cmd.ExecuteNonQuery();
        this.DialogResult = DialogResult.OK;
        Close();
    } catch (Exception ex) {
        MessageBox.Show("Ошибка добавления/изменения строки \n" +
            ex.Message);
        return;
    }
}

```

Задание:

Добавить в приложение формы просмотра и редактирования справочной таблицы

Содержание отчета:

- 1) схема БД
- 2) скриншот основного меню главной формы;
- 3) скриншот формы редактирования;
- 4) программный код метода загрузки данных;
- 5) программный код метода удаления строки;
- 6) программный код формы редактирования справочника;
- 7) скриншот формы просмотра справочника с заполненными данными;
- 8) скриншоты тестирования ситуаций с ошибками в процессе добавления, редактирования таблицы справочника.